中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术

实验项目编号 2 实验名称 简单功能型处理器设计

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- **设计**步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、 设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 LaTeX 中)

1 单周期 MIPS/RISC-V 处理器设计

本实验旨在设计并实现一个支持 MIPS32 指令集子集的单周期处理器。为保证系统的精确执行, 我采用了自顶向下的硬件设计思想, 将 CPU 架构拆解为若干功能模块, 并分别对其内部逻辑与接口规范进行设计。

程序计数器(PC)模块负责指令地址的寄存与更新, 其设计原理基于简单的时序逻辑, 在每个时钟上升沿捕获并输出新的指令地址。为了支持复杂的控制流转向, 我在这里设计了多级地址选择网络: 除了默认的顺序执行通路 PCnext 外, 还有与控制单元及 ALU 协同, 计算并选择由条件分支指令(Branch)引发的相对偏移地址, 以及由跳转指令(Jump)触发的绝对地址, 从而确保不同指令场景下的 PC 数值更新。

控制单元构成了整个处理器的"大脑", 在这里我采用纯组合逻辑进行实现。核心设计思路是通过提取输入机器指令的 opcode 和 func 字段进行联合译码, 将复杂的 MIPS 指令集聚类映射为有限的若干种宏类型。基于这些宏类型, 控制单元会向全局派生出互相协同的控制信号(如 ALUSrc、MemToReg、RegDst、MemWrite 等)。这些信号负责控制数据在各级多路选择器与功能部件间的流向。

为满足复杂的数据访存需求, 我的设计思路是通过提取 ALU 计算出的物理内存地址低两位, 结合具体的加载/存储指令类型(如 lwl、lwr 等), 动态生成精确到字节级别的写使能掩码(Write_strb)以及多路数据拼接网络, 实现了对复杂且不规则内存数据布局的访存功能。

在算术逻辑单元控制信号(ALUop)的生成机制上, 我选择了通过指令类型(type)与特定机器码字段(func 或 opcode)直接按位拼接的组合逻辑。控制器通过对指令类型进行归类, 将 R 型算术指令、I 型立即数运算、访存以及分支指令进行并行条件判断, 直接提取指令的低位特征码以生成 3 位的 ALU_op 操作码。

寄存器堆(Register File)的控制逻辑重点在于多路写回数据与目标地址的精确选择。在写入地址(RF_waddr)的选择上, 我选择通过 RegDst 信号动态区分 R 型指令的目标寄存器(rd)与 I 型指令的目标寄存器(rt), 并特

意为链接跳转指令(jal)开辟了硬连线至 31 号寄存器的独立通路。在写入数据(RF_wdata)的控制上,我设计了一个结构庞大的多路选择器,根据指令类型精准选择写入的数据。此外,写使能信号(RF_wen)的生成经过了严密的条件约束,不仅涵盖了常规的算术与访存写回,还巧妙融合了条件传送指令(如 movz、movn)的动态状态位(Zero),确保了寄存器状态更新的绝对安全与时序准确。

基于以上划分,我的 CPU 结构大致如下图所示:

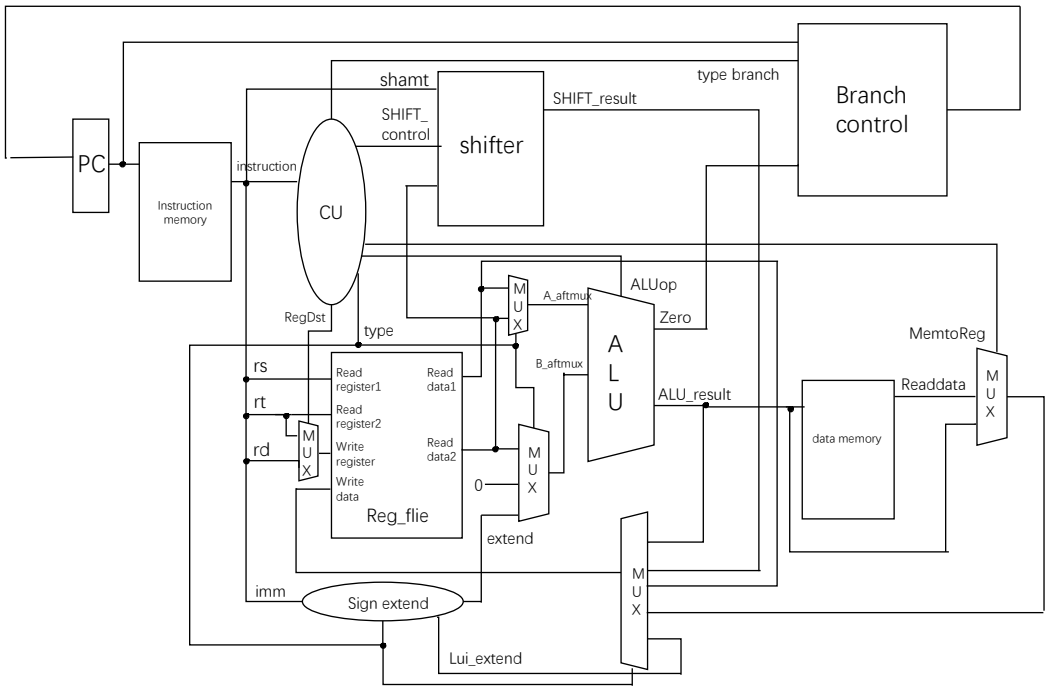


图 1: CPU 架构

2 其他关键模块设计(shifter/ALU)

shifter 需要根据输入命令对两个操作数进行相应的移位操作,我的核心设计思路是在 CPU 确定输入操作数的形式,在 shifter 内部根据 func 的不同选择输出不同的结果数。

ALU 用于新添加了集中操作形式,需要进行运算方式的扩展。逻辑运算的扩展思路较为简单,直接添加几个逻辑运算的线段即可实现操作;而无符号数比较则可以利用 Carryout 的设计思路进行编排,比较过程中使用 co 的值进行快速判断即可。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 单周期 MIPS/RISC-V 处理器设计

• **指令译码与分类逻辑**。机器码的解析是控制流生成的第一步。我首先提取了 32 位 Instruction 中的操作码(opcode)、寄存器索引(rs,rt,rd)、移位量(shamt)以及立即数(imm)等关键底层字段。利用逻辑掩码与按位或操作,将繁杂的 MIPS 指令集高效映射为 11 种独立互斥的指令类型状态(如 R_Alu、I_Branch、I_Load 等)。这种并行组合逻辑译码方式,并非穷举,也没有使用 always 语句,符合老师的要求。

```
//Instruction Decode
wire [5:0]  opcode      = Instruction[31:26];
wire [4:0]  rs          = Instruction[25:21];
wire [4:0]  rt          = Instruction[20:16];
wire [4:0]  rd          = Instruction[15:11];
wire [4:0]  shamt       = Instruction[10:6];
wire [5:0]  func        = Instruction[5:0];
wire [4:0]  base        = Instruction[25:21];
wire [4:0]  REG         = Instruction[20:16];
wire [15:0] imm         = Instruction[15:0];
wire [25:0] Instr_index = Instruction[25:0];



wire [3:0] type =       {4{opcode == `SPECIAL && func[5] == 1'b1}} & `R_Alu |
                        {4{opcode == `SPECIAL && func[5:3] == 3'b000}} & `R_Shift |
                        {4{opcode == `SPECIAL && {func[5:3], func[1]} == 4'b0010}} & `R_Jump |
                        {4{opcode == `SPECIAL && {func[5:3], func[1]} == 4'b0011}} & `R_Move |
                        {4{opcode == 6'b000001}} & `REGIMM |
                        {4{opcode[5:1] == 5'b00001}} & `JType |
                        {4{opcode[5:2] == 4'b0001}} & `I_Branch |
                        {4{opcode[5:3] == 3'b001 && opcode != 6'b001111}} & `I_Alu |
                        {4{opcode == 6'b001111}} & `lui |
                        {4{opcode[5:3] == 3'b100}} & `I_Load |
                        {4{opcode[5:3] == 3'b101}} & `I_Store;
```

图 2: 译码分类部分代码

• **算术逻辑与立即数扩展控制**。ALU 控制逻辑的构建上，控制信号直接复用译码阶段生成的宏类型进行派生。ALU_op 信号通过多路条件运算符，将 R 型指令的 func 字段与 I 型算术指令的 opcode 字段重新编码为统一的 3 位底层操作码，驱动 ALU 执行加减法或逻辑运算。同时，为保障数据运算的位宽一致性，立即数扩展逻辑不仅内建了常规的符号扩展（sign_extend）与零扩展（zero_extend），还特设了针对 lui 指令的 16 位左移拼接逻辑。控制器会根据具体的访存或逻辑运算需求，选通正确的 32 位拓展结果进入 ALU 的源操作数 B 端口（B_aftmux）。

```
//ALU control
wire [2:0] ALU_op =     {{3{type == `R_Alu && func[3:2] == 2'b00}} & {func[1], 2'b10}}|//addu,subu
                        {{3{type == `R_Alu && func[3:2] == 2'b01}} & {func[1], 1'b0, func[0]}}|//and,or,xor,nor
                        {{3{type == `R_Alu && func[3:2] == 2'b10}} & {~func[0], 2'b11}}|//slt,sltu
                        {{3{type == `I_Alu && opcode[2:0] == 3'b001}} & {opcode[1], 2'b10}}|//addiu
                        {{3{type == `I_Alu && opcode[2] == 1'b1}} & {opcode[1], 1'b0, opcode[0]}}|//andi,ori,xori
                        {{3{type == `I_Alu && opcode[2:1] == 2'b01}} & {~opcode[0], 2'b11}}|//slti,sltiu
                        {{3{type == `R_Move}} & `ALUop_SUB}|//movz,movn
                        {{3{type == `REGIMM}} & `ALUop_SLT}|//bltz,bgez
                        {{3{type == `I_Branch && opcode[1] == 1'b0}} & `ALUop_SUB}|//beq, bne
                        {{3{type == `I_Branch && opcode[1] == 1'b1}} & `ALUop_SUB}|//blez,bgtz
                        {{3{type == `I_Load || type == `I_Store}} & `ALUop_ADD};//lw,sw

wire [31:00] A_aftmux = (type == `R_Move)? RF_rdata2: RF_rdata1;
wire [31:00] B_aftmux = {{32{ALUSrc}}&extend}|
                        {{32{type ==`R_Alu || type == `I_Branch}} & RF_rdata2}|
                        {{32{type == `R_Move || type == `REGIMM}} & 32'b0};
```

图 3: 算术逻辑部分代码

```
//imm extended control
wire [31:0] sign_extend         = {{16{imm[15]}}, imm};
wire [31:0] zero_extend         = {16'b0, imm};
wire [31:0] offset_two_extend   = {{14{imm[15]}}, imm, 2'b0};
assign lui_extend               = {imm, 16'b0};
assign extend                   = (((type == `I_Alu)&&(ALU_op!={opcode[1], 1'b0, opcode[0]}))||
                                  (type == `I_Load)||(type == `I_Store))?sign_extend:zero_extend;
```

图 4: 立即数扩展部分代码

• **访存控制**。该部分是开发中最为困难的环节。面对不同大小(字、半字、字节)的内存访问需求,代码通过截取 ALU 计算所得物理地址的低两位(Addr_low),进行控制逻辑的构造。在 Load 控制逻辑中,我实现了标准的数据位宽截取,针对 lwl 与 lwr 指令开发了复杂的位掩码拼接电路,将内存中的非对齐数据与寄存器原始数据(RF_rdata2)进行拼接。在 Store 控制逻辑中,组装了写入数据(Write_data),并同步生成了写使能掩码信号(Write_strb),适配了 swl、sb 等指令时的非边界对齐写入。

```
//load control
assign  Address         =       {ALU_result[31:2],2'b00};
wire [1:0]  Addr_low    =       ALU_result[1:0];
wire [7:0]  byte_read   =       (Addr_low == 2'b00)?Read_data[7:0]:
                                (Addr_low == 2'b01)?Read_data[15:8]:
                                (Addr_low == 2'b10)?Read_data[23:16]:
                                                    Read_data[31:24];
wire [15:0] half_read   =       (Addr_low[1])?Read_data[31:16]:
                                             Read_data[15:0];
wire [31:0] lwl_read    =       (Addr_low == 2'b00)?{Read_data[7:0],RF_rdata2[23:0]}:
                                (Addr_low == 2'b01)?{Read_data[15:0],RF_rdata2[15:0]}:
                                (Addr_low == 2'b10)?{Read_data[23:0],RF_rdata2[7:0]}:
                                                    {Read_data[31:0]};
wire [31:0] lwr_read    =       (Addr_low == 2'b00)?Read_data[31:0]:
                                (Addr_low == 2'b01)?{RF_rdata2[31:24],Read_data[31:8]}:
                                (Addr_low == 2'b10)?{RF_rdata2[31:16],Read_data[31:16]}:
                                                    {RF_rdata2[31:8],Read_data[31:24]};
assign      load_result =       (opcode == 6'b100000)?{{24{byte_read[7]}}, byte_read}://lb
                                (opcode == 6'b100100)?{24'b0, byte_read}://lbu
                                (opcode == 6'b100001)?{{16{half_read[15]}}, half_read}://lh
                                (opcode == 6'b100101)?{16'b0, half_read}://lhu
                                (opcode == 6'b100011)?Read_data://lw
                                (opcode == 6'b100010)?lwl_read://lwl
                                (opcode == 6'b100110)?lwr_read://lwr
                                                      32'b0;
```

图 5: Load 逻辑部分代码

```
//store control
assign Write_strb       =       (opcode == 6'b101011)?4'b1111: //sw
                                (opcode == 6'b101001)?((Addr_low == 2'b00)?4'b0011:4'b1100): //sh
                                (opcode == 6'b101000)? ((Addr_low == 2'b00)?4'b0001:
                                                        (Addr_low == 2'b01)?4'b0010:
                                                        (Addr_low == 2'b10)?4'b0100:
                                                                           4'b1000)://sb
                                (opcode == 6'b101010)? ((Addr_low == 2'b11)?4'b1111:
                                                        (Addr_low == 2'b10)?4'b0111:
                                                        (Addr_low == 2'b01)?4'b0011:
                                                                           4'b0001)://swl
                                (opcode == 6'b101110)? ((Addr_low == 2'b11)?4'b1000:
                                                        (Addr_low == 2'b10)?4'b1100:
                                                        (Addr_low == 2'b01)?4'b1110:
                                                                           4'b1111)://swr
                                                                           4'b0000;
assign Write_data       =       (opcode == 6'b101011)? RF_rdata2://sw
                                (opcode == 6'b101001)? ((Addr_low == 2'b00)?{16'b0,RF_rdata2[15:0]}:
                                                        {RF_rdata2[15:0],16'b0}):
                                (opcode == 6'b101000)? ((Addr_low == 2'b00)?{24'b0,RF_rdata2[7:0]}:
                                                        (Addr_low == 2'b01)?{16'b0,RF_rdata2[7:0],8'b0}:
                                                        (Addr_low == 2'b10)?{8'b0,RF_rdata2[7:0],16'b0}:
                                                        {RF_rdata2[7:0], 24'b0}):
                                (opcode == 6'b101010)? ((Addr_low == 2'b11)?RF_rdata2[31:0] :
                                                        (Addr_low == 2'b10)?{8'b0,  RF_rdata2[31:8]}:
                                                        (Addr_low == 2'b01)?{16'b0, RF_rdata2[31:16]}:
                                                                           {24'b0, RF_rdata2[31:24]}):
                                (opcode == 6'b101110)? ((Addr_low == 2'b11)?{RF_rdata2[7:0], 24'b0}:
                                                        (Addr_low == 2'b10)?{RF_rdata2[15:0], 16'b0}:
                                                        (Addr_low == 2'b01)?{RF_rdata2[23:0], 8'b0} :
                                                                            RF_rdata2[31:0]):// swr
                                                       32'b0;
```

图 6: Store 逻辑部分代码

• **移位器与条件传送控制**。移位控制逻辑根据 R_Shift 类型与指令特征，在常量移位量（shamt）与寄存器内移位量（rs 寄存器低 5 位）之间进行选通，生成动态移位位宽（shiftbit），并联合控制码交由 shifter 模块处理。与此同时，利用 ALU 实时输出的零标志位（Zero），构建了条件传送指令（movz 与 movn）的判定开关（Mov_true）。该逻辑作为控制信号，当状态寄存器满足零或非零的先决条件时，直接激活寄存器堆的写使能信号。

```
//shifter control
wire [4:0] shiftbit      = {{5{type == `R_Shift && func[5:2] == 4'b0000 }} & shamt} |
                           {{5{type == `R_Shift && func[5:2] == 4'b0001}} & RF_rdata1[4:0]};
wire [1:0] SHIFT_control = func[1:0];
shifter shifter_module(RF_rdata2, shiftbit, SHIFT_control, SHIFT_result);


//move control
wire Mov_true = (type == `R_Move)&((func[0]==1'b0 & Zero)|(func[0]==1'b1 & ~Zero));
```

图 7: 移位器与条件传送控制部分代码

• **寄存器堆控制**。在寄存器堆控制层面，写入地址（RF_waddr）与写入数据（RF_wdata）均采用了多级条件选择器。其中，常规指令的结果依据 RegDst 路由至目标寄存器，而针对函数链接调用（jal）则硬连线至 31 号寄存器，并将保存的返回地址置为 PC + 8。写使能信号（RF_wen）严格把控了所有的合法寄存操作。

```
//reg_file control
assign RF_wdata =       (type == `R_Alu || type == `I_Alu)? ALU_result:
                        (type == `R_Shift)? SHIFT_result:
                        (opcode == 6'b000011 || (type == `R_Jump && func == 6'b001001))? PC + 8:
                        (type == `I_Load)? load_result:
                        (type == `R_Move)? RF_rdata1:lui_extend;
assign RF_waddr         = (opcode == 6'b000011)? 31:((RegDst)? rd:rt);
assign RF_raddr1        = rs;
assign RF_raddr2        = (type == `REGIMM)? 5'b0:rt;
assign RF_wen = ( (type == `R_Alu)||(type == `R_Shift)||((type == `R_Jump)&&func[0])||
                        ((type == `JType) && opcode[0])||(type == `I_Alu)||(type == `I_Load)||(type == `lui)||Mov_true);
reg_file reg_file_module(
        .clk(clk),
        .waddr(RF_waddr),
        .raddr1(RF_raddr1),
        .raddr2(RF_raddr2),
        .wen(RF_wen),
        .wdata(RF_wdata),
        .rdata1(RF_rdata1),
        .rdata2(RF_rdata2)
        );
```

图 8: 寄存器堆控制部分代码

• **分支判定与指令获取**。分支控制逻辑(isbranch)融合了等值比较(beq/bne)与基于 ALU 符号位反馈的高级零点比较(blez、bgtz、bltz 等),当且仅当比较条件与指令操作码完全吻合时,方才触发分支使能。在底层的时序逻辑块中,程序计数器(PC)数据来源根据当前指令类型与分支判定结果,在顺序步进的下一条地址 PC + 4、通过偏移计算得到的分支目标地址(branch_PC),以及绝对地址拼接产生的跳转目标(jumpaddr)这三条硬件通路中进行切换。

```
//branch control
wire isbranch = ((type ==  `I_Branch) & (opcode[1:0] == 2'b00) & Zero)|
                ((type ==  `I_Branch) & (opcode[1:0] == 2'b01) & ~Zero)|
                ((type ==  `I_Branch) & (opcode[1:0] == 2'b10) & (ALU_result[31]|Zero))|
                ((type ==  `I_Branch) & (opcode[1:0] == 2'b11) & (~ALU_result[31]&~Zero))|
                ((type == `REGIMM) & (REG[0] == 0) & ALU_result[0])|
                ((type == `REGIMM) & (REG[0] == 1) & ~ALU_result[0]);


//Instruction Fetch
wire [31:0] PC_next     = PC + 4;
wire [31:0] branch_PC   = PC_next + offset_two_extend;
wire [31:0] jumpaddr    = (type == `R_Jump)?RF_rdata1:
                          (type == `JType)?{PC_next[31:28],Instr_index,2'b00}:32'b0;
always @(posedge clk) begin
        if (rst) begin
                PC <= 32'b0;
        end
        else begin
                PC <= (type == `R_Jump || type == `JType) ? jumpaddr:((Branch & isbranch)?branch_PC:PC_next);
        end
end
```

图 9: 分支判定与指令获取控制部分代码

完成上述模块定义后,进行测试,产生的波形图如下图所示:

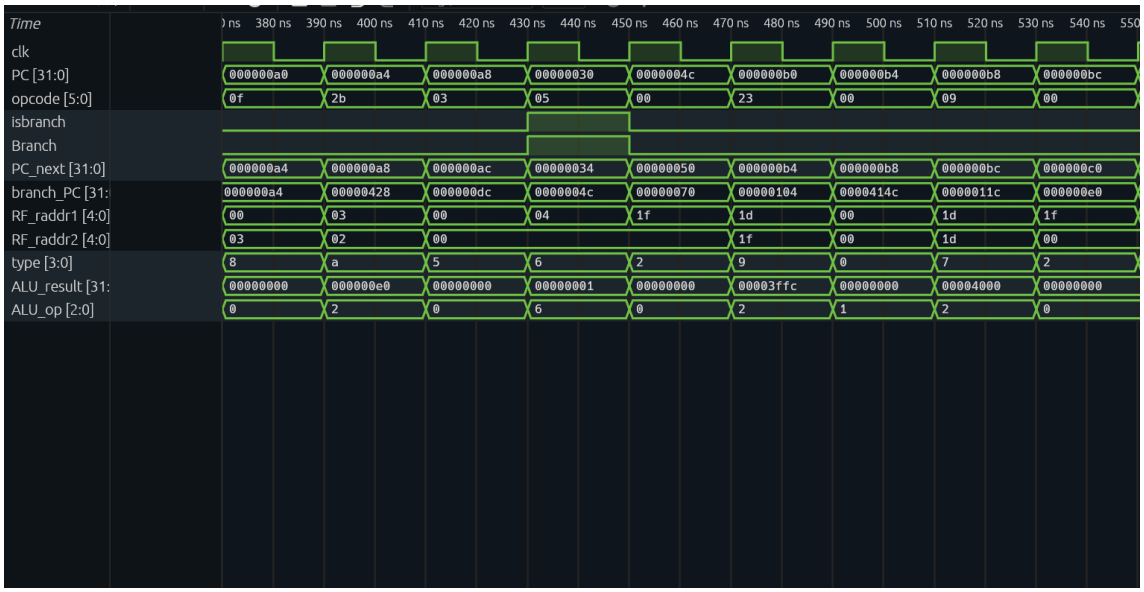


图 10: 波形图

可以看到,根据指令内容的不同,各指令协调运行,正常工作。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

- **单周期 MIPS/RISC-V 处理器问题补充。**

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	RTL 语法	所有提前使用未定义多位宽 wire 型数据的模块	没有考虑多位宽 wire 型变量需要先定义再使用	Verilog 语法不熟悉	提前定义了多位宽 wire 型变量
2	逻辑电路设计	写使能信号控制	需要写使能信号为 1 时,写使能信号为 0	少写了几种写使能的指令类型	将缺失的指令类型补写
3	行为仿真	blez 指令实现	写错了 blez 涉及的跳转判断,使 PC 跳转错误	分支指令模块设计环节出现选错标志位问题	修改了控制逻辑
4	行为仿真	写掩码控制	写反了写掩码的控制逻辑,使写入数据错误	store 型指令操作机制理解不清	将写掩码逻辑改回来了

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

虽然单周期 CPU 实现了所有指令 CPI = 1 的极佳指标,但在开发最复杂的 lw 指令时,我直观地感受到了访存指令对系统频率的掣肘。lw 指令需要依次经过取指、译码、ALU 地址计算、数据存储器读取和寄存器写回,这条漫长的组合逻辑延迟决定了整个 CPU 时钟周期的下限,导致主频无法提升。这一痛点让我深刻理解了流水线(Pipeline)技术以及多周期 CPU 存在及实现的必要性。

类别	名称	体会	后续计划
设计方法	顶层与数据通路设计	掌握了自顶向下的模块化划分。但在复杂访存(如 swl/swr)设计初稿中，由于对硬件掩码物理机制理解不透彻,导致控制逻辑反转。	在后续架构设计中，针对复杂核心模块，先充分理解指令实际操作内容后再进行代码映射。
RTL 编写	组合逻辑与译码实现	熟练运用位拼接与逻辑掩码实现了高效并行译码。但初期常忽略严谨的线网定义规范 (如多位宽隐式声明引发报错)。	严格遵循“先声明后使用”规范，强化对边界条件的收尾检查。
验证方法	行为级仿真与调试	能够通过波形逆向追踪数据流，精准定位了 blez 跳转标志位和 store 写掩码的逻辑漏洞。	学习其它有效的 debug 方法，丰富自己的技能库。

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- **思考题 1:ALUop 的编码有什么规律？表格中的 ALUop 编码是否有优化空间？**
  算术类 (ADD/SUB):ALUop 均以 10 结尾(010, 110)。
  逻辑类 (AND/OR/XOR/NOR):ALUop 的中间位均为 0,即以 00 或 01 结尾。
  比较类 (SLT/SLTU):ALUop 均以 11 结尾(111, 011)。
  我认为可以根据 func 的末三位直接决定控制位,必要时可以加入 func[3] 作为分类依据

五、 实验时长

在课后,你花费了大约 15 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	2	无
开发	10	花费了较多时间完善每一条指令
调试	3	调试过程较为顺利

表 3: 实验时长